SoloPhysics: From a Mass-Spring Prototype to a 3D GLAD Renderer

An Obsidian + LaTeX Development Handbook for a Python Programmer Learning C++

SoloPhysics development notes

31 October 2026

Contents

1 Project Goals and Chronology **1**
- 1.1 Motivation . . . 1
- 1.2 Development log — 28 to 31 October 2026 . . . 1
- 1.3 Why the order mattered . . . 2
- 1.4 Fundamental architectural choices . . . 2
- 1.5 Roadmap decision log . . . 2

2 Environment, Project Layout and Build **3**
- 2.1 Hardware and operating environment . . . 3
- 2.2 File structure . . . 3
- 2.3 Compilation mental model (from Python to C++) . . . 3
- 2.4 Git safeguards . . . 4

3 C++ for a Python Developer **5**
- 3.1 The biggest differences . . . 5
- 3.2 From source to symbols . . . 5
- 3.3 Values, references, const and pointers . . . 5
- 3.4 Aggregates and brace initialisation . . . 6
- 3.5 Iteration and index types . . . 6
- 3.6 Automatic type deduction, templates and structured bindings . . . 6
- 3.7 Lambdas and capture syntax . . . 6
- 3.8 Optional values . . . 7
- 3.9 Classes, public/private, constructors and cleanup . . . 7
- 3.10 Error handling and ownership . . . 7
- 3.11 Syntax you will meet in the renderer . . . 7
- 3.12 Pointer/reference traps for Python developers . . . 8

4 Core Data Model and Source Architecture **9**
- 4.1 Core model . . . 9
- 4.2 `Vector3D.hpp`: descriptive alias . . . 9
- 4.3 `Node.hpp`: mass particle . . . 10
- 4.4 `Beam.hpp`: connected spring/damper . . . 10
- 4.5 `PhysicsWorld`: responsible for one step . . . 10
- 4.6 Why separate responsibilities? . . . 11

5 Forces, Springs and Damping **12**
- 5.1 Newton’s second law . . . 12
- 5.2 The direction of a beam . . . 12
- 5.3 Hooke’s law plus axial damping . . . 12
- 5.4 One-dimensional worked calculation . . . 13
- 5.5 Spring energy and damping . . . 13

5.6 What is not modelled yet? . . . 13

6 Numerical Integration, Stability and Timesteps 14
6.1 Continuous versus discrete motion . . . 14
6.2 Choosing a small step . . . 14
6.3 The frame-time accumulator . . . 14
6.4 Critical damping and oscillations . . . 15
6.5 Numerical checklist . . . 15

7 First Verification: Suspended Mass 16
7.1 Experiment design . . . 16
7.2 Predict the resting height *before* simulation . . . 16
7.3 Damping experiment . . . 16
7.4 Lessons recorded . . . 16
7.5 Suggested regression assertion . . . 16

8 Square Drop, Impact and Resting-Contact Fix 18
8.1 From one spring to a connected structure . . . 18
8.2 Baseline free fall . . . 18
8.3 The micro-bouncing bug . . . 18
8.4 Why the final upper height was below 1 m . . . 19

9 Plastic Deformation, Fracture and Damage Flags 20
9.1 The three regimes . . . 20
9.2 Recorded plastic update . . . 20
9.3 Important limitation: moving strain reference . . . 20
9.4 Actual v0.4 demonstration parameters . . . 20
9.5 Colours and state priority . . . 21
9.6 First damage debugging incident . . . 21

10 The First 2D GLFW/OpenGL Renderer 22
10.1 Why add graphics only after headless tests? . . . 22
10.2 Basic GLFW lifecycle . . . 22
10.3 Legacy OpenGL prototype . . . 22
10.4 The early screen-space/world-space trap . . . 22
10.5 Why replace the renderer? . . . 23

11 Interactive Controls, Picking and Mouse Springs 24
11.1 The problem: force accumulators were reset . . . 24
11.2 The virtual mouse spring . . . 24
11.3 Picking: from metres to pixels . . . 24
11.4 3D dragging is not a 2D screen coordinate . . . 25
11.5 UI feedback . . . 25

12 From Square to 3D Cube and Orbit Camera 26
12.1 The v0.4 geometry . . . 26
12.2 Orbit camera geometry . . . 26
12.3 From world coordinates to GPU pixels . . . 26
12.4 Screen-space picking and unprojection . . . 27
12.5 Scope of 3D collisions . . . 27

13 Modern OpenGL, GPU Resources and GLSL 28
13.1 Why the OpenGL 3.3 change matters . . . 28
13.2 Two shader stages . . . 28
13.3 Compile and link shaders . . . 28
13.4 The packed vertex layout . . . 28
13.5 Per-frame rendering . . . 29
13.6 Why OpenGL cleanup matters . . . 29
13.7 Render order and depth . . . 29
13.8 Future graphics improvements . . . 30

14 The GLEW Failure and GLAD 2 Migration **31**
14.1 What actually happened on 31 October 2026 . . . 31
14.2 GLEW vs GLAD . . . 31
14.3 GLAD 2, not GLAD 1 . . . 31
14.4 Current CMake method . . . 31
14.5 Debug checklist . . . 32

15 Line-by-Line CMake and Linking Guide **33**
15.1 The actual supplied `CMakeLists.txt` . . . 33
15.2 Decode it one line at a time . . . 33
15.3 CMake language syntax is not C++ . . . 34
15.4 Compile versus link errors . . . 34
15.5 Rebuild best practices . . . 34

16 Real Errors, Diagnostics and Lessons **35**
16.1 Debugging timeline . . . 35
16.2 Reproducible debugging funnel . . . 35
16.3 What `ninja: no work to do` actually means . . . 36
16.4 Warnings versus errors . . . 36
16.5 Useful SoloLinux commands . . . 36
16.6 The rule: measure before changing . . . 36

17 Tests, Learning Exercises and Worked Answers **37**
17.1 A. Regression test matrix . . . 37
17.2 B. Exercise 1 — Translate Python to C++ . . . 37
17.3 C. Exercise 2 — Calculate spring force . . . 37
17.4 D. Exercise 3 — Interpret the lambda . . . 38
17.5 E. Exercise 4 — Derive static equilibrium . . . 38
17.6 F. Exercise 5 — Why 28 beams? . . . 38
17.7 G. Exercise 6 — Debug a missing force . . . 38
17.8 H. Exercise 7 — Change OpenGL safely . . . 38
17.9 I. Suggested exercise roadmap . . . 38

18 Current Limits and a Path Toward Vehicle Simulation **39**
18.1 Present capabilities (confirmed by source/conversation) . . . 39
18.2 Not yet established . . . 39
18.3 Suggested engineering order . . . 39
18.4 Research questions to answer experimentally . . . 39
18.5 Source discipline . . . 40

19 Websites, Sources and Attribution **41**
19.1 Provenance statement — important . . . 41
19.2 Direct project evidence . . . 41
19.3 Official documentation and educational sources . . . 41
19.4 Licence/attribution awareness . . . 42
19.5 Sources mentioned in original planning but not implemented . . . 42

20 C++ Syntax Cheat Sheet for Python Programmers **43**
20.1 Read this before opening `Renderer.cpp` . . . 43
20.2 Special vector syntax . . . 44
20.3 Common traps . . . 44
20.4 Further reading . . . 44

1 Project Goals and Chronology

1.1 Motivation

SoloPhysics began on **28 October 2026** with a long-term objective of a BeamNG.drive-style deformable vehicle simulator, built independently to learn C++, mathematical modelling and engine development. The important design decision was to build a **headless physics core**

first, verify it numerically, and add visualisation only after the results made sense. Inspiration from node–beam vehicle physics is supported by BeamNG's official explanation; the present project is an original prototype, not a recreation of the full BeamNG solver.

1.2 Development log — 28 to 31 October 2026

Date	Milestone	Evidence / outcome
28 Oct	Select C++20, GLM, CMake, Ninja, Git; headless-first node–beam design	Development discussion
28–29 Oct	Create `Node`, `Beam`, `PhysicsWorld` and gravity / spring forces / integration	Reviewed code fragments and user tests
29 Oct	Single suspended mass experiment; vary stiffness/damping and plasticity	Equilibrium near **1.9019 m** verified
30 Oct	Four-node, six-beam square dropped onto ground	Position and velocity output supplied
30 Oct	Replace small-contact restitution with resting threshold	Micro-bounce changed to nearly zero velocity
30 Oct	Add 2D GLFW/OpenGL renderer	User confirmed window worked
30 Oct	Add camera controls, pause/reset/gravity toggles, mouse-force dragging	User confirmed drag worked
30 Oct	Enable yielding/fracture, highlight nodes and draw drag spring	User confirmed interactive damage worked
31 Oct	Replace 2D square with 3D cube, OpenGL 3.3 shaders, orbit camera	Source bundle v0.4 verified
31 Oct	Initial 3D GLEW loader failed on user's setup; migrate to GLAD 2	User confirmed GLAD version worked

> **Note:** Version labels v0.1–v0.4 are teaching/milestone labels used during our development conversations, not a claim about Git tags or formal releases. Exact commit IDs for the SoloPhysics project were not provided here.

1.3 Why the order mattered

```
flowchart TD
 A[Physics data model] --> B[Force accumulation]
 B --> C[Time integration]
 C --> D[Hanging mass verification]
 D --> E[Square + collision]
 E --> F[Stable resting contacts]
 F --> G[2D live renderer]
 G --> H[Interactive input + damage]
 H --> I[3D camera + cube]
 I --> J[GLAD loader migration]
 J --> K[Future vehicle chassis]
```

The sequence prevents rendering bugs from being confused with simulation bugs. For example, the 2D CLI output already proved the nodes were moving before any shaders were written. Likewise, terminal damage counts can distinguish physics failure from incorrect beam colouring.

1.4 Fundamental architectural choices

- **Simulation is separate from graphics:** the renderer receives a `const PhysicsWorld&`; it reads state but does not integrate equations.
- **Real units:** positions in metres, masses in kilograms, acceleration in m/s², force in newtons; spring coefficient in N/m, damping in N·s/m.
- **Deterministic-size steps:** `1/240` s in interactive v0.3–v0.4. This is not a claim of fully deterministic cross-platform behaviour.
- **Explicit topology:** beams store *indices*, not owning pointers to nodes, so the world can store nodes in a `std::vector`.
- **Per-beam failure state:** `broken` prevents the spring from transmitting force; `plasticallyDeformed` records historical deformation for display.

1.5 Roadmap decision log

Initially chosen: straightforward mass–spring–damper with semi-implicit Euler. **Deferred:** FEM, XPBD, constrained contacts, self-collision, spatial acceleration structures, full vehicle dynamics, solver calibration and multithreading. Their absence is an intentional scope boundary rather than evidence they are solved.

See 18 - Limits and Future Roadmap and 19 - Bibliography and Attribution.

2 Environment, Project Layout and Build

2.1 Hardware and operating environment

The project is developed on **SoloLinux**, an Arch-derived Linux setup, using a C++20 compiler (g++ in the recorded build logs), GLFW, GLM, Mesa/OpenGL, CMake, Ninja and later GLAD 2. The user was running a Hyprland/Wayland desktop when the GLEW loader problem was investigated.

```
sudo pacman -S --needed git python python-jinja glfw glm mesa cmake ninja
cd ~/Projects/SoloPhysics
cmake -S . -B build-glad -G Ninja
cmake --build build-glad
./build-glad/solo_physics
```

- `sudo pacman -S --needed`: installs only missing packages.
- `cmake -S .`: source directory is here.
- `-B build-glad`: binary/build directory is a separate location.
- `-G Ninja`: choose Ninja as the build-system generator.
- `cmake --build`: execute that generator with dependency checking.
- &&: run the next command **only if the previous command succeeded**; prevents launching a stale executable after a failed compile.

2.2 File structure

```
SoloPhysics/
├── CMakeLists.txt
├── include/solophysics/
│   ├── Vector3D.hpp
│   ├── Node.hpp
│   ├── Beam.hpp
│   ├── PhysicsWorld.hpp
│   └── Renderer.hpp
├── src/
│   ├── main.cpp
│   ├── physics/PhysicsWorld.cpp
│   └── rendering/Renderer.cpp
└── build-glad/       # generated; do not edit by hand
```

Headers (`.hpp`) describe interfaces and shared types. **Implementation files** (`.cpp`) define functions. `main.cpp` assembles the demo and owns the application loop. A larger engine could split tests into a separate target and move the engine into a library.

2.3 Compilation mental model (from Python to C++)

Python often imports a module at runtime. C++ typically uses a **preprocessor**, **compiler**, **linker**, then a native executable:

```
flowchart LR
 A[main.cpp] --> C[C++ compiler]
 B[Renderer.hpp / PhysicsWorld.hpp] --> C
 D[Renderer.cpp] --> E[C++ compiler]
 F[PhysicsWorld.cpp] --> G[C++ compiler]
 C --> H[Object files]
 E --> H
 G --> H
 H --> I[Linker]
 J[GLFW / GLAD] --> I
 I --> K[solo_physics]
```

`#include <solophysics/Renderer.hpp>` effectively makes declarations available to the current translation unit. It is **not** a runtime package import. A missing declaration causes a compile

error; a declaration without a matching compiled definition can cause a linker error.

2.4 Git safeguards

```
git status --short
git add CMakeLists.txt include src
git commit -m "Document current working GLAD renderer"
git log --oneline --max-count=10
```

These are example commands; the exact history and branch state depend on your repository. Commit before replacing generated upgrade files. See 16 - Errors, Diagnostics and Lessons.

3 C++ for a Python Developer

3.1 The biggest differences

C++ is statically typed, compiled ahead of execution, and exposes object lifetime, copies, references, pointers, stack/heap allocation and generic templates more directly than Python. The SoloPhysics code uses **C++20**, but many constructs predate C++20.

Concept	Python	SoloPhysics C++
Variable	`mass = 1.0`	`float mass = 1.0f;`
Constant-like binding	`DT = 1 / 240` (convention)	`constexpr double dt = 1.0 / 240.0;`
Container	`nodes = []`	`std::vector<Node> nodes;`
Fixed-size collection	`positions = (...)`	`std::array<Vector3D, 8> positions;`
Optional result	None or index	`std::optional<std::size_t>`
Function	`def step(dt):`	`void step(float dt);`
Exception	`raise ValueError(...)`	`throw std::runtime_error(...);`
Object attribute	`node.position`	`node.position` or `nodePtr->position`
Module organisation	`physics_world.py`	`PhysicsWorld.hpp` + `PhysicsWorld.cpp`

> **Note:** Python analogy limitations Python names usually refer to objects; C++ values can be copied by assignment. `Node a = b;` copies b. `Node& a = b;` is an alias to b. `Node* p = &b;` is a pointer containing its address.

3.2 From source to symbols

```cpp
#include <glm/geometric.hpp>
namespace solophysics {
    using Vector3D = glm::vec3;
}
```

- `#include` tells the preprocessor to make declarations available.
- `namespace solophysics { ... }` groups symbols, rather like `solophysics.` names in Python.
- `::` is the **scope resolution operator**: `glm::vec3` refers to `vec3` in namespace `glm`.
- `using Vector3D = glm::vec3;` defines a type alias, **not a new class**.
- `;` terminates most statements. `{}` delimit blocks instead of Python indentation.

3.3 Values, references, const and pointers

```cpp
void inspect(const PhysicsWorld& world) {
    const Node& a = world.nodes[0];
    // a.mass = 5.0f; // ERROR: read-only reference
}

void accelerate(Node& node, const Vector3D& force) {
    node.force += force; // changes the original node
}
```

`PhysicsWorld& world` means **reference** (alias, no copy). `const PhysicsWorld&` makes the accessed object read-only from this function. `Node* node` means **pointer**. `&node` in an expression takes an address, and `*ptr` dereferences a pointer. The `*` in `std::optional` dereferences the contained value too, but `optional` is not itself a pointer.

```
GLFWwindow* window = nullptr;
glfwMakeContextCurrent(window);
```

`nullptr` is the null-pointer literal; GLFW manages a window behind this pointer. **Never** dereference a null pointer. The renderer destroys the GLFW window during cleanup.

3.4 Aggregates and brace initialisation

```
Node topLeft;
topLeft.position = {-0.5f, 3.0f, 0.0f};
topLeft.mass = 1.0f;
```

`3.0f` is a `float` literal, while `3.0` is usually a `double`. `{x, y, z}` supplies elements/constructor inputs to a GLM vector. `Vector3D{0.0f}` constructs a zero vector (as used in the project).

3.5 Iteration and index types

```
for (Beam& beam : world.beams) {
    beam.breakStrain = 0.30f;
}

for (std::size_t i = 0; i < world.nodes.size(); ++i) {
    const Node& node = world.nodes[i];
}
```

The first is a **range-based `for`** (like Python's `for beam in beams`); `&` avoids copying each `Beam` and permits mutation. The second is an indexed loop; `std::size_t` is an unsigned size/index type, `++i` increments it, `world.nodes[i]` indexes the vector. The `i` variable exists **only inside the loop**—the exact scope error that appeared in the project's 2D mouse-picking code.

3.6 Automatic type deduction, templates and structured bindings

```
const std::array<std::pair<std::size_t, std::size_t>, 28> connections{{
    {0, 1}, {1, 2} /* ... other entries ... */
}};

for (const auto& [a, b] : connections) {
    world.addBeam(a, b, stiffness, damping);
}
```

- `std::array<T, N>`: fixed-size sequence with `N` elements of type `T`.
- `std::pair<A,B>`: two-value aggregate, like a typed 2-tuple.
- `auto`: compiler infers the type; unlike Python, the type is still static.
- `const auto& [a, b]`: structured binding, unpacking the pair **without copying the pair**.
- `<...>`: template arguments, not a less-than comparison in this context.

3.7 Lambdas and capture syntax

```
auto createWorld = []() {
    PhysicsWorld world;
    return world;
};

auto applyMouseForce = [&](PhysicsWorld& physics) {
    if (!grabbedNode) return;
    Node& node = physics.nodes[*grabbedNode];
    node.force += mouseStiffness * (mouseTarget - node.position)
                - mouseDamping * node.velocity;
```

```
};
```

A lambda is an anonymous function (Python `lambda` analogy, but C++ lambdas may contain multiple statements). `[]` is the capture list. Empty `[]`: use no outside local variables. `[&]`: capture used enclosing local variables by **reference**. The function body begins after `{`. `return` indicates the result; a `void` lambda has no returned value. The user's callback is stored in `std::function<void(PhysicsWorld&)>` and invoked from the physics timestep. **Lifetime rule:** captured references must remain valid for as long as the callback uses them.

3.8 Optional values

```
std::optional<std::size_t> grabbedNode;
if (grabbedNode) {
    Node& node = world.nodes[*grabbedNode];
}
grabbedNode.reset();
```

`std::optional<T>` is an explicit 'value or no value' wrapper, similar to `int | None` in Python typing. `if (grabbedNode)` checks **presence**, not whether the index is nonzero; index `0` is valid. `*grabbedNode` retrieves it only after checking presence. `.reset()` returns to the empty state.

3.9 Classes, public/private, constructors and cleanup

```
class Renderer {
public:
    Renderer(int width, int height);
    ~Renderer();
    Renderer(const Renderer&) = delete;
private:
    GLFWwindow* window = nullptr;
};
```

- `class` defaults to private members, `struct` defaults to public members.
- `public:` exposes methods to callers; `private:` hides implementation details.
- `Renderer(...)` is a **constructor**, called on object creation.
- `~Renderer()` is a **destructor**, called during destruction.
- `= delete` prohibits copying the renderer; two copies owning one GLFW window could double-free GPU/window resources.
- RAII (*Resource Acquisition Is Initialization*) ties cleanup to object lifetime. Python has analogous context managers (`with`), but destruction timing differs.

3.10 Error handling and ownership

```
try {
    Renderer renderer(1200, 800);
    // application loop
} catch (const std::exception& e) {
    std::cerr << "SoloPhysics error: " << e.what() << '\n';
    return 1;
}
```

The `catch` catches standard-library exception types derived from `std::exception`. `std::cerr` is the error output stream; `<<` inserts text into the stream. Objects with automatic lifetime clean up as the exception propagates. Constructors must themselves clean up resources they acquired if construction throws before completion.

3.11 Syntax you will meet in the renderer

Syntax	Meaning
`GLenum` / `GLuint`	OpenGL API-defined integer-like types
`sizeof(Vertex)`	Byte size of a vertex structure
`offsetof(Vertex, colour)`	Byte offset of member `colour` inside `Vertex`
`reinterpret_cast<void*>(...)`	Explicit low-level cast for the OpenGL vertex offset API
`R"GLSL( ... )GLSL"`	Raw string literal containing shader source
`const char*`	Pointer to a null-terminated character sequence
`static_cast<float>(x)`	Explicit checked-category conversion, e.g. `double` to `float`
`std::clamp(x, low, high)`	Bound value within interval
`std::nullopt`	Empty optional value
`#pragma once`	Widely supported header multiple-inclusion protection
`constexpr`	Compile-time evaluable constant when applicable

3.12 Pointer/reference traps for Python developers

1. A reference cannot be reseated after binding. `Node& a = nodes[0]` will always refer to that node object during its lifetime.
2. `std::vector` can reallocate on growth, invalidating references/pointers to elements. This is why a beam stores indices, with explicit bounds checks.
3. `const` is enforced by the compiler in ordinary code; it isn't a runtime immutable proxy.
4. Ownership of C pointers to OpenGL/GLFW resources must be managed explicitly; don't assume Python-style garbage collection.
5. A `std::function` holding `[&]` captures can create dangling references if its lifetime exceeds those local variables.

See 20 - C++ Quick Reference for an exam-style sheet and 17 - Verification Tests and Exercises for practice.

4 Core Data Model and Source Architecture

4.1 Core model

The simulation is a graph: **nodes are vertices**, and **beams are undirected edges**. Each node has state (mass, position, velocity, accumulated force); each beam refers to two node indices and describes its constitutive behaviour.

```
classDiagram
class Node {
  +Vector3D position
  +Vector3D velocity
  +Vector3D force
  +float mass
  +bool fixed
}
class Beam {
  +size_t nodeA
  +size_t nodeB
  +float restLength
  +float stiffness
  +float damping
  +float yieldStrain
  +float breakStrain
  +float plasticity
  +bool broken
  +bool plasticallyDeformed
}
class PhysicsWorld {
  +vector~Node~ nodes
  +vector~Beam~ beams
  +Vector3D gravity
  +step(dt)
  +addNode(node)
  +addBeam(a,b,k,c)
}
class Renderer {
  +render(world,...)
  +pickNode(world,radius)
  +updateCamera()
}
PhysicsWorld "1" o-- "many" Node
PhysicsWorld "1" o-- "many" Beam
Renderer ..> PhysicsWorld : reads const reference
```

4.2 `Vector3D.hpp`: descriptive alias

Recorded design:

```cpp
#pragma once
#include <glm/vec3.hpp>
namespace solophysics {
    using Vector3D = glm::vec3;
}
```

Interpretation: `Vector3D` and `glm::vec3` denote the same underlying type. GLM supplies vector addition, scalar multiplication, dot products and lengths. Using an alias makes a future math-library replacement easier conceptually, though replacing GLM would require more work than just the alias.

4.3 Node.hpp: mass particle

Recorded prototype (illustrative reconstruction of the earlier header):

```
struct Node {
    Vector3D position{0.0f};
    Vector3D velocity{0.0f};
    Vector3D force{0.0f};
    float mass = 1.0f;
    bool fixed = false;
};
```

Units: `position` m, `velocity` m/s, `force` N, `mass` kg. A fixed node should not be numerically integrated. The `force` field stores the **sum of forces for the current timestep**, not a permanent applied force.

4.4 Beam.hpp: connected spring/damper

Recorded interface (field ordering may differ locally):

```
struct Beam {
    std::size_t nodeA = 0;
    std::size_t nodeB = 0;
    float restLength = 0.0f;
    float stiffness = 1000.0f;
    float damping = 10.0f;
    float yieldStrain = 0.15f;
    float breakStrain = 0.60f;
    float plasticity = 2.0f;
    bool plasticDeformationEnabled = true;
    bool plasticallyDeformed = false;
    bool broken = false;
};
```

The beam has no separate mass or bending DOF. `nodeA`, `nodeB` are **indices** into `world.nodes`. `restLength` is the length at which the spring's elastic component is zero. Later plastic deformation mutates that value, so original-length-based comparisons would require a separate original-length field.

4.5 PhysicsWorld: responsible for one step

The previously recorded step sequence:

```
void PhysicsWorld::step(float dt) {
    // timestep validation
    validateState();
    resetForces();
    applyExternalForces();
    if (externalForceCallback) externalForceCallback(*this);
    solveBeamForces();
    integrate(dt);
    detectAndResolveGroundContacts();
    updateDamage(dt);
    validateState();
}
```

This is a *recorded conceptual sequence*, not a byte-for-byte snapshot of the newest local `PhysicsWorld.cpp`. Its order has physical consequences: using forces from the current positions for integration; applying ground corrections after integration; and evaluating breakage after movement means a breaking beam may still have contributed a force during the immediately preceding timestep.

4.6 Why separate responsibilities?

`main.cpp` constructs a test **scenario**, processes UI events and calls physics/render. `PhysicsWorld.cpp` evolves the physical state. `Renderer.cpp` converts the latest state to GPU draw commands. As a result, the same physics core could be called by command-line tests, a 2D renderer, a 3D renderer or (later) a headless benchmark.

Mental model from Python: `PhysicsWorld` is the domain model, `Renderer` is a view, and `main.cpp` is the controller/application entry point. Unlike a typical pure-Python application, the C++ declarations, definitions, template instantiations and linker all have to agree.

Read alongside source snapshots `Source Snapshots/v0.4-GLAD/src/main.cpp` and `.../Renderer.cpp`. See also 05 - Forces and Spring Dynamics and 15 - CMake Deep Dive.

5 Forces, Springs and Damping

5.1 Newton's second law

For a node with constant mass m:

$$\sum \mathbf{F} = m\mathbf{a}, \qquad \mathbf{a} = \frac{\sum \mathbf{F}}{m}.$$

The world starts each timestep by clearing accumulated node forces, then adding external and internal forces:

```
node.force = Vector3D{0.0f};
if (!node.fixed) node.force += node.mass * gravity;
```

If $m = 1\,\text{kg}$ and $g_y = -9.81\,\text{m/s}^2$, the gravitational force is $F_y = -9.81\,\text{N}$. A mass of 2 kg experiences $-19.62\,\text{N}$, but *the same gravitational acceleration* when no other forces apply.

5.2 The direction of a beam

For nodes A and B:

$$\begin{aligned} \mathbf{d} &= \mathbf{x}_B - \mathbf{x}_A, \\ L &= \|\mathbf{d}\|, \\ \hat{\mathbf{d}} &= \frac{\mathbf{d}}{L}, \\ e &= L - L_0. \end{aligned}$$

Here L is current length, L_0 is the **current rest length**, and e is extension in metres. A very small L must be guarded before dividing by it (`if (length < 1e-6f) continue;`).

5.3 Hooke's law plus axial damping

The force on node **A** in the project's implementation is

$$\boxed{\mathbf{F}_A = \Big[k(L - L_0) + c\big((\mathbf{v}_B - \mathbf{v}_A)\cdot\hat{\mathbf{d}}\big)\Big]\hat{\mathbf{d}}}.$$

The force on B is $\mathbf{F}_B = -\mathbf{F}_A$ (Newton's third law). Positive extension pulls A toward B. If B moves away from A along the beam, the damping contribution opposes the relative motion by pulling A toward B and B toward A. Verify the sign *relative to the endpoint*; changing the sign of damping is a common instability bug.

Recorded logic:

```
Vector3D delta = b.position - a.position;
float length = glm::length(delta);
if (length < 1e-6f) continue;
Vector3D direction = delta / length;
float extension = length - beam.restLength;
float relativeVelocity = glm::dot(b.velocity - a.velocity, direction);
float springForce = beam.stiffness * extension;
float dampingForce = beam.damping * relativeVelocity;
Vector3D force = (springForce + dampingForce) * direction;
if (!a.fixed) a.force += force;
if (!b.fixed) b.force -= force;
```

Units: k N/m; c N·s/m; $L - L_0$ m; $\dot{L}$ m/s; both resulting terms are N. These beams transmit **axial** load only, not moments or direct bending. Cross-bracing creates network-level resistance to shear.

5.4 One-dimensional worked calculation

Take $L_0 = 1.00$ m, $L = 1.03$ m, $k = 1000$ N/m, $c = 10$ N·s/m, and $\dot{L} = 0.2$ m/s:

$$F = k(0.03) + c(0.2) = 30 + 2 = 32\,\text{N}.$$

The magnitude is 32 N. Node A experiences $+32\hat{\mathbf{d}}$ and node B $-32\hat{\mathbf{d}}$. If the beam is compressing, extension and possibly axial velocity are negative, reversing the sign.

5.5 Spring energy and damping

An ideal undamped Hookean beam has potential energy:

$$U_s = \tfrac{1}{2}k(L - L_0)^2.$$

The axial damper dissipates energy at instantaneous rate

$$P_d = -c\dot{L}^2 \leq 0,$$

assuming the velocity projection and force signs above. This is a useful test for damping implementation. **Caveat:** node ground clamping, plastic changes to rest length and explicit integration alter the simple conservative-energy budget.

5.6 What is not modelled yet?

- No rotation or moment constraints at beam endpoints.
- No finite cross-section, distributed mass or bending stiffness for a single beam.
- No continuous collision detection or self-collision.
- No force-calibrated fatigue or crash material model.
- No physically derived plastic yield surface; current thresholds are intentionally simplified.

See 09 - Plasticity, Fracture and Damage and 18 - Limits and Future Roadmap. BeamNG's node–beam explanation is useful conceptual background but does not imply Solver equivalence.

6 Numerical Integration, Stability and Timesteps

6.1 Continuous versus discrete motion

Newton's laws are differential equations:

$$\frac{d\mathbf{x}}{dt} = \mathbf{v}, \qquad \frac{d\mathbf{v}}{dt} = \mathbf{a}(\mathbf{x}, \mathbf{v}, t).$$

The CPU computes states at discrete times t_n. SoloPhysics uses **semi-implicit Euler**, also called symplectic Euler in this context:

$$\boxed{\begin{aligned}\mathbf{v}_{n+1} &= \mathbf{v}_n + \Delta t\, \mathbf{a}_n,\\ \mathbf{x}_{n+1} &= \mathbf{x}_n + \Delta t\, \mathbf{v}_{n+1}.\end{aligned}}$$

Contrast explicit Euler: $\mathbf{x}_{n+1} = \mathbf{x}_n + \Delta t\, \mathbf{v}_n$. Using the *new* velocity for the position step generally behaves better for simple oscillators but does **not** make arbitrarily stiff springs stable.

```
Vector3D acceleration = node.force / node.mass;
node.velocity += acceleration * dt;
node.position += node.velocity * dt;
```

A `fixed` node skips these updates. Nonpositive/NaN mass requires validation before division.

6.2 Choosing a small step

At $\Delta t = 1/240$ seconds, the simulation attempts 240 updates for every simulated second. The single hanging-mass test initially used $1/120$ seconds; the later interactive application used $1/240$. A window rendering at 60 frames per second normally receives about four physics steps per frame.

A rough scale for a 1 kg body on a 1000 N/m spring is

$$\omega_n = \sqrt{k/m} = \sqrt{1000} \approx 31.62\,\text{rad/s},$$

$$\omega_n \Delta t \approx 0.132 \quad (240\,\text{Hz}).$$

Small $\omega_n \Delta t$ generally improves numerical behaviour. Coupled beams have multiple vibration modes, so one scalar spring estimate is **not** a proof of overall stability. Damping too high can also cause numerical trouble with explicit schemes.

6.3 The frame-time accumulator

Current v0.4 `main.cpp` includes:

```
constexpr double fixedDt = 1.0 / 240.0;
double previousTime = glfwGetTime();
double accumulator = 0.0;

while (!renderer.shouldClose()) {
    double now = glfwGetTime();
    double frameTime = std::clamp(now - previousTime, 0.0, 0.1);
    previousTime = now;
    accumulator += frameTime;
    while (accumulator >= fixedDt) {
        world.step(static_cast<float>(fixedDt));
        accumulator -= fixedDt;
```

```
    }
    renderer.render(world, grabbedNode, hoveredNode, mouseTarget);
}
```

This excerpt omits actual input processing, pause branch, and diagnostics for clarity. The *actual* source retains them. `glfwGetTime()` returns elapsed time; `std::clamp` limits a single frame to at most 0.1 s of physics catch-up. That means a lengthy stall can cause simulated time to fall behind real elapsed time. The accumulator preserves unsimulated fractional time within the chosen frame-time cap.

Important: fixed-step does not guarantee fully deterministic results across all hardware, floating-point modes or different input-event timings. The app currently samples mouse input once per render loop; it does not record/replay a timestamped input stream.

6.4 Critical damping and oscillations

For an isolated 1D mass–spring–damper:

$$m\ddot{x} + c\dot{x} + kx = 0,$$

$$\omega_n = \sqrt{\frac{k}{m}}, \qquad c_{crit} = 2\sqrt{km}, \qquad \zeta = \frac{c}{2\sqrt{km}}.$$

- $\zeta < 1$: underdamped, oscillates while decaying.
- $\zeta = 1$: critically damped in the *ideal isolated system*.
- $\zeta > 1$: overdamped.

For $m = 1$ kg, $k = 100$ N/m, $c_{crit} = 20$ N·s/m. With $c = 5$, $\zeta = 0.25$; with $c = 0.6$, $\zeta = 0.03$. These idealised formulas are excellent for validating the hanging-mass test but don't transfer directly to every mode of a 28-beam network.

6.5 Numerical checklist

1. Validate finite and positive masses and finite positions/velocities.
2. Avoid normalising zero-length beams.
3. Monitor maximum speed/force/strain if nodes explode or vanish.
4. Apply forces **after** resetting accumulators.
5. Ensure each action uses a known fixed Δt.
6. Use controlled tests with known answers before a complex vehicle.

Additional reading: Glenn Fiedler, Fix Your Timestep!. See 07 - The Hanging Mass Experiment.

7 First Verification: Suspended Mass

7.1 Experiment design

The first successful validation used a **single 1 kg mass** suspended from a fixed anchor at $y = 3$ m by a beam of initial length $L_0 = 1$ m, stiffness $k = 100$ N/m and damping initially about 5 N·s/m. Gravity pointed down with magnitude 9.81 m/s².

```
Anchor at y=3.0000 m   [fixed]
            |
            | beam, L0=1 m
            |
Mass initially y=2.0000 m [1 kg]
            ↓ gravity
```

7.2 Predict the resting height *before* simulation

At equilibrium, the spring must exert $mg = 9.81$ N upward. The spring therefore extends by

$$\Delta L = \frac{mg}{k} = \frac{1 \times 9.81}{100} = 0.0981\,\text{m}.$$

Its settled length is $L = 1.0981$ m. Because the mass is suspended below the anchor,

$$\boxed{y_{eq} = 3.0 - 1.0981 = 1.9019\,\text{m}}.$$

The actual CLI output converged to approximately **1.9019 m**, matching this prediction at the reported precision. This gave initial evidence that gravity, axial spring direction and integration were coherently implemented.

7.3 Damping experiment

Keeping $m = 1$, $k = 100$ and changing damping from roughly $c = 5$ to $c = 0.6$ caused oscillations to persist longer, as expected from $\zeta = c/(2\sqrt{km})$. A key experimental precaution was to disable plastic deformation (`plasticDeformationEnabled=false`) and raise `breakStrain` so the spring's rest length would not change during this elastic baseline test.

> **Note:** Why this is a strong engineering test A test is most useful when you can calculate an expected result without relying on the program. Otherwise, incorrect code may still draw a convincing animation.

7.4 Lessons recorded

- Be sure to save `main.cpp` before rebuilding; an earlier test accidentally ran with unchanged source and misleadingly similar output.
- Print damping, stiffness, plasticity state and rest length alongside results.
- Compare analytical equilibrium, not just whether a mass visually stops moving.
- Damping should affect oscillatory transient behaviour, not ideal static equilibrium while rest length is fixed.

7.5 Suggested regression assertion

```cpp
// Illustrative test only: assumes you set up the above world and
// run it long enough to settle without plasticity.
const float expectedY = 1.9019f;
const float tolerance = 0.003f;
assert(std::abs(world.nodes[massNodeIndex].position.y - expectedY) < tolerance);
```

This illustrative assertion is not an existing project test. It shows how to automate the observation. See 17 - Verification Tests and Exercises.

8 Square Drop, Impact and Resting-Contact Fix

8.1 From one spring to a connected structure

The 2D test replaced the suspended node with four nodes placed in the XY plane:

Node	Initial position (m)
0, top left	$(-0.5, 3, 0)$
1, top right	$(0.5, 3, 0)$
2, bottom left	$(-0.5, 2, 0)$
3, bottom right	$(0.5, 2, 0)$

Six beams connected four perimeter edges and two crossing diagonals. Diagonal cross-bracing stops the square becoming an unconstrained pin-jointed parallelogram. With $k = 1000$ N/m, $c = 10$ N·s/m and each node of mass 1 kg, the shape began falling from rest.

```
flowchart TB
 A((0)) --- B((1))
 A --- C((2))
 B --- D((3))
 C --- D
 A --- D
 B --- C
```

(The diagram is connectivity, not the actual screen positions.)

8.2 Baseline free fall

Before contact, all four nodes experienced almost identical downward motion. At $t = 0.25$ s, the reported speed was approximately $v_y = -2.4525$ m/s, matching $-gt = -9.81 \times 0.25$.

At $t = 0.50$ s, $v_y = -4.9050$ m/s and bottom nodes were at $y = 0.7635$ m. These are valuable baseline checks: internal beams should not create a net external force on an initially undeformed freely falling assembly.

The bottom starts at 2 m, so continuous ideal free-fall predicts first contact near

$$t_{impact} = \sqrt{2h/g} = \sqrt{4/9.81} \approx 0.639\,\text{s}.$$

The logged data at 0.75 s already showed impact, rebound and compression.

8.3 The micro-bouncing bug

The original ground solver clamped below-ground positions to $y = 0$ and multiplied downward velocity by a restitution coefficient. At rest, gravity repeatedly caused tiny penetration; the solver repeatedly reversed velocity. Earlier results showed bottom nodes at $y = 0$ but with persistent $v_y \approx +0.0136$ m/s. That is an **artefact** of the contact correction, not physical perpetual bouncing.

A simple resting threshold was introduced:

```
constexpr float restingVelocityThreshold = 0.2f;
if (node.position.y < groundY) {
    node.position.y = groundY;
    if (node.velocity.y < 0.0f) {
        float impactSpeed = -node.velocity.y;
        node.velocity.y = impactSpeed < restingVelocityThreshold
                          ? 0.0f
```

```
                        : -node.velocity.y * restitution;
    }
    node.velocity.x *= (1.0f - friction);
    node.velocity.z *= (1.0f - friction);
}
```

The displayed post-fix results settled to bottom $y = 0$, $v_y = 0$ (at print precision) and upper $y \approx 0.9926$ m by approximately 2.5 s. This was successful for the test.

> **Note:** Not a physically complete contact solver This is a position clamp and velocity heuristic. It does not compute persistent normal impulses, static friction constraints, or continuous collision detection. Multiplying tangential velocity by `(1 - friction)` each contact step is framerate/step-dependent and not Coulomb friction. Do not treat it as a production crash solver.

8.4 Why the final upper height was below 1 m

At rest, the bottom row sits at $y = 0$ while the upper row is around 0.9926 m, slightly lower than the square's original 1 m height. The network's axial springs are carrying gravitational loads and compressing. The exact load distribution depends on beam topology and constraints, so do not apply the single-hanging-mass formula blindly to the square.

This experiment validated a multi-node topology, elastic loads, impact response and the importance of separating numerical artefacts from genuine material behaviour.

9 Plastic Deformation, Fracture and Damage Flags

9.1 The three regimes

1. **Elastic:** beam stretches/compresses then returns toward its rest length when the load disappears.
2. **Plastic:** beam's rest length itself changes; deformation persists when forces stop.
3. **Fractured:** beam is marked broken and its force contribution is skipped thereafter.

SoloPhysics implements a simplified **strain-threshold** model. For current length L and current rest length L_0,

$$\varepsilon = \frac{L - L_0}{L_0}, \qquad \varepsilon_{abs} = |\varepsilon|.$$

When $|\varepsilon| \geq \varepsilon_{break}$, `beam.broken=true`. Otherwise, if plastic deformation is enabled and $|\varepsilon| > \varepsilon_y$, the code gradually moves L_0 toward a yield-boundary length.

9.2 Recorded plastic update

```
float factor = std::clamp(beam.plasticity * dt, 0.0f, 1.0f);
float yieldBoundary = beam.restLength *
    (1.0f + std::copysign(beam.yieldStrain, strain));
float excess = length - yieldBoundary;
float oldRestLength = beam.restLength;
beam.restLength += excess * factor;
beam.restLength = std::max(beam.restLength, 1e-6f);
if (std::abs(beam.restLength - oldRestLength) > 1e-7f)
    beam.plasticallyDeformed = true;
```

This is a simplification recorded during the earlier 2D implementation; verify the newest `PhysicsWorld.cpp` before changing it. `std::copysign` keeps tension and compression yield boundaries on the appropriate side of zero strain. `std::clamp` limits how far the rest length can move during one step.

9.3 Important limitation: moving strain reference

Because strain is divided by **the updated rest length**, yielding can reduce current strain and can postpone fracture during slow pulling. A more rigorous model should distinguish:

$$\varepsilon_{total} = \frac{L - L_{original}}{L_{original}}, \qquad \varepsilon_{elastic} = \frac{L - L_{plastic\,rest}}{L_{plastic\,rest}}$$

and track a history variable for accumulated plastic strain, damage or dissipated plastic work. A stress/force-based yield model and material calibration would be better for vehicle physics.

9.4 Actual v0.4 demonstration parameters

From the checked 3D `src/main.cpp` bundle:

Variable	Demo setting	Purpose
`stiffness`	650 N/m	Elastic axial response
`damping`	12 N·s/m	Axial viscous damping
`yieldStrain`	0.06	Plastic update triggers above 6%
`breakStrain`	0.30	Beam fractures above 30% current-rest strain

Variable	Demo setting	Purpose
`plasticity`	0.6	Rest-length adaptation rate
`mouseStiffness`	500 N/m	User's virtual drag spring
`mouseDamping`	35 N·s/m	Drag force damper
`maxMouseForce`	1500 N	Clamp for large mouse pulls

These **aren't validated properties of steel, aluminium or vehicle parts**; they were chosen for an interactive demo.

9.5 Colours and state priority

The GLAD renderer uses green for near rest length, blue for compression, red for extension, orange for a plastically deformed beam, and grey for broken. Plastic history takes priority over current elastic strain (so an orange beam can also currently be stretched or compressed). Broken beams are still drawn as grey straight lines between endpoints **for debugging**, not as dangling fragments.

9.6 First damage debugging incident

At one point, the square visibly responded to dragging but never yielded or broke because `main.cpp` still contained:

```
beam.plasticDeformationEnabled = false;
beam.breakStrain = 100.0f;
```

The fix was to enable yielding, set feasible demonstration thresholds, anchor one node to force actual structural strain, and print `Plastic beams/Broken beams` alongside maximum strain. The user later confirmed the complete interactive damage test worked.

See 16 - Errors, Diagnostics and Lessons and 18 - Limits and Future Roadmap.

10 The First 2D GLFW/OpenGL Renderer

10.1 Why add graphics only after headless tests?

The CLI square-drop already produced useful positions and velocities. A renderer made the dynamics understandable visually, but it shouldn't change the physics results. The first GLFW version received `const PhysicsWorld&` and used an **orthographic XY projection** of a 3D physics state.

10.2 Basic GLFW lifecycle

```
if (!glfwInit()) throw std::runtime_error("GLFW init failed");
GLFWwindow* window = glfwCreateWindow(1000, 700, "SoloPhysics", nullptr, nullptr);
if (!window) throw std::runtime_error("Window creation failed");
glfwMakeContextCurrent(window);
glfwSwapInterval(1);
while (!glfwWindowShouldClose(window)) {
    glfwPollEvents();
    // step physics and draw
    glfwSwapBuffers(window);
}
glfwDestroyWindow(window);
glfwTerminate();
```

This is an illustrative lifecycle excerpt, **not** the complete current code. GLFW creates native windows and OpenGL contexts; it does not itself implement the physics engine or GPU shaders. `glfwSwapInterval(1)` requests vsync; actual frame rate depends on environment.

10.3 Legacy OpenGL prototype

The original `Renderer.cpp` used a requested **OpenGL 2.1 compatibility context**, `glBegin(GL_LINES)`, `glVertex2f(...)`, `glEnd()`, `glMatrixMode`, and `glOrtho`. This was an intentionally short teaching step, not the final architecture. Each frame drew:

- ground line;
- all unbroken beams as coloured segments;
- nodes as points.

```
glBegin(GL_LINES);
for (const Beam& beam : world.beams) {
    if (beam.broken) continue;
    const Node& a = world.nodes[beam.nodeA];
    const Node& b = world.nodes[beam.nodeB];
    glVertex2f(a.position.x, a.position.y);
    glVertex2f(b.position.x, b.position.y);
}
glEnd();
```

Syntax: `glBegin` and `glEnd` are function calls. `GL_LINES` is a named API constant. The `for (const Beam& beam : ...)` loop accesses each beam by read-only reference. The renderer reads node positions and translates them into screen geometry.

10.4 The early screen-space/world-space trap

World position x, y is measured in metres; mouse location is measured in window pixels. A screen-centred orthographic camera with vertical span H converts cursor coordinates approximately as:

$$x_w = x_c + \left(2\frac{x_p}{W} - 1\right)\frac{H}{2}\frac{W}{H_{pixel}},$$

$$y_w = y_c + \left(1 - 2\frac{y_p}{H_{pixel}}\right)\frac{H}{2}.$$

The actual prototype also handled framebuffer size separately from window size for scaling on HiDPI setups. Its zoom and panning affected the mapping.

10.5 Why replace the renderer?

Modern OpenGL core profiles intentionally omit legacy immediate-mode drawing calls. Moving to OpenGL 3.3 shaders and vertex buffers makes the engine more portable to modern contexts and introduces the GPU pipeline required for larger meshes and later vehicle rendering.

See 13 - Modern OpenGL and GLSL and GLFW’s official getting-started guide.

11 Interactive Controls, Picking and Mouse Springs

11.1 The problem: force accumulators were reset

The first drag attempt failed because the application added forces at the wrong point in the update sequence. Each `PhysicsWorld::step()` begins by setting all `node.force` fields to zero. If `main.cpp` writes `node.force += ...` **before** `step()`, that force disappears on the very next reset.

Solution: `PhysicsWorld` exposes an `externalForceCallback` invoked **after** resetting forces and applying gravity but **before** solving beams and integrating.

```
// Recorded interface
std::function<void(PhysicsWorld&)> externalForceCallback;

// In step() (illustrative excerpt)
resetForces();
applyExternalForces();
if (externalForceCallback) externalForceCallback(*this);
solveBeamForces();
integrate(dt);
```

`std::function<void(PhysicsWorld&)>` means: an object callable with one mutable `PhysicsWorld&` and returning `void`. Python analogy: a stored callable attribute such as `world.external_force_callback = callback`.

11.2 The virtual mouse spring

$$\boxed{\mathbf{F}_{mouse} = k_m(\mathbf{x}_{target} - \mathbf{x}_{node}) - c_m\mathbf{v}_{node}}.$$

The current 3D source contains:

```
auto applyMouseForce = [&](PhysicsWorld& physics) {
    if (!grabbedNode || *grabbedNode >= physics.nodes.size()) return;
    Node& node = physics.nodes[*grabbedNode];
    if (node.fixed) return;
    Vector3D force = mouseStiffness * (mouseTarget - node.position)
                   - mouseDamping * node.velocity;
    constexpr float maxMouseForce = 1500.0f;
    float magnitude = glm::length(force);
    if (magnitude > maxMouseForce)
        force *= maxMouseForce / magnitude;
    node.force += force;
};
```

This code **does not teleport a node**; it applies a bounded spring force, allowing the beam network to respond. `node.force += ` adds to rather than replaces gravity and other external forces.

11.3 Picking: from metres to pixels

The original 2D picking selected the nearest node inside a fixed world-space radius (initially 0.2 m, later enlarged for diagnosis). This felt inconsistent as zoom changed.

The current 3D renderer uses `glm::project(world.nodes[i].position, viewMatrix(), projectionMatrix(), viewport)` to transform each node into framebuffer coordinates. It compares distance to the mouse cursor **in pixels** (19 px in the supplied v0.4 caller), so the selection radius is consistent with screen scale.

```
auto hoveredNode = renderer.pickNode(world, 19.0f);
if (left && !previousLeft) {
    grabbedNode = hoveredNode;
```

```
    if (grabbedNode && world.nodes[*grabbedNode].fixed)
        grabbedNode.reset();
    if (grabbedNode)
        renderer.beginDrag(world.nodes[*grabbedNode].position);
}
```

`left && !previousLeft` is an **edge detector**: act once when the button transitions from released to pressed. Without it, selection could restart every frame while held.

11.4 3D dragging is not a 2D screen coordinate

In 3D, the cursor represents a **ray** extending out from the camera. The program unprojects the near/far cursor points, forms a ray, then intersects it with a plane passing through the selected node and facing the camera:

$$\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}, \quad \mathbf{n} \cdot (\mathbf{r}(t) - \mathbf{p}_0) = 0,$$

$$\boxed{t = \frac{\mathbf{n} \cdot (\mathbf{p}_0 - \mathbf{o})}{\mathbf{n} \cdot \mathbf{d}}}.$$

If the denominator is close to zero, the ray is parallel to the plane; return no target (`std::nullopt`). This gives intuitive movement approximately parallel to the screen at the node's initial depth. It does not implement arbitrary depth-axis gizmos yet.

11.5 UI feedback

- Normal node: yellow; fixed node: purple.
- Hovered: cyan; grabbed: orange.
- A yellowish drag-spring line joins grabbed node and cursor target.
- Space, R, G and later F toggle pause, reset, gravity and the anchor; Esc exits.

Debugging lesson: seeing `Left mouse clicked` confirmed GLFW, but wasn't proof that node picking succeeded; only seeing selected IDs and applying forces confirmed the chain.

Further reading: GLFW Input Guide. See 03 - C++ for a Python Developer for lambdas and optional values.

12 From Square to 3D Cube and Orbit Camera

12.1 The v0.4 geometry

The actual supplied v0.4 `main.cpp` creates **8 nodes**, four at $y = 2$ m and four at $y = 3$ m, with $x, z \in \{-0.5, +0.5\}$ m. Node 0 starts fixed. Gravity starts off, so the user can pull the structure without free fall; pressing F releases the anchor, and G enables gravity.

The edges form a fully connected 8-node network, K_8, with

$$\binom{8}{2} = \frac{8 \times 7}{2} = 28 \text{ unique connections.}$$

The source enumerates 12 cube edges, 12 face diagonals and 4 body diagonals. This is deliberately heavily braced; it is not a realistic vehicle-body finite element model.

```
const std::array<Vector3D, 8> positions{{
    {-0.5f, 2.0f, -0.5f},
    { 0.5f, 2.0f, -0.5f},
    { 0.5f, 2.0f,  0.5f},
    {-0.5f, 2.0f,  0.5f},
    {-0.5f, 3.0f, -0.5f},
    { 0.5f, 3.0f, -0.5f},
    { 0.5f, 3.0f,  0.5f},
    {-0.5f, 3.0f,  0.5f}
}};
```

`std::array<Vector3D,8>` has exactly eight GLM vectors. The *two sets* of braces are common in `std::array` aggregate initialisation and should not be confused with a Python nested tuple of arbitrary size.

12.2 Orbit camera geometry

The camera stores a **target** (point to look at), a **yaw** (left/right rotation), a **pitch** (up/down rotation), and a **distance**. Given camera target **t**, distance r, yaw θ and pitch ϕ:

$$\mathbf{x}_{camera} = \mathbf{t} + r \begin{bmatrix} \cos\phi \sin\theta \\ \sin\phi \\ \cos\phi \cos\theta \end{bmatrix}.$$

The renderer calls `glm::lookAt(cameraPosition(), cameraTarget, {0,1,0})` to obtain a view matrix. Right-drag changes yaw/pitch, middle-drag translates the target, scrolling scales distance (clamped between 1.5 and 40 in the supplied version).

12.3 From world coordinates to GPU pixels

A 3D point in homogeneous coordinates $\mathbf{p} = (x, y, z, 1)$ passes through

$$\boxed{\mathbf{p}_{clip} = PVM\mathbf{p}_{model}}.$$

Current app geometry is directly in world coordinates, so its model transform M is effectively identity and the code computes `mvp = projectionMatrix() * viewMatrix()`. The matrix product order matters: for column-vector convention, view acts first and projection acts second.

The current projection is a perspective camera:

```
glm::perspective(glm::radians(55.0f), aspect, 0.05f, 100.0f);
```

55° is vertical field of view; `aspect = framebufferWidth/framebufferHeight`; `0.05` m and `100` m set near/far clipping planes. Keeping the near plane above zero avoids an invalid perspective projection.

12.4 Screen-space picking and unprojection

- `glm::project`: world point → framebuffer X/Y/depth.
- `glm::unProject`: framebuffer X/Y/depth → world point.
- `cursorRay()`: two unprojected endpoints → a ray.
- `beginDrag()`: record a point and a camera-facing plane normal.
- `mouseDragTarget()`: ray/plane intersection.

This is **selection by projected point radius**, not mesh triangle ray-intersection. It is appropriate for a node-based prototype but would need occlusion, depth sorting and hit tests for complex visible geometry.

12.5 Scope of 3D collisions

The existing solver checks node height versus a flat $y = groundY$ plane. The renderer draws a **grid on the XZ plane**; the grid does not itself simulate collision. The simple ground solver handles point contacts, not edge/triangle collisions, contact patches, angular friction or full cube–cube intersection.

See 13 - Modern OpenGL and GLSL and 18 - Limits and Future Roadmap.

13 Modern OpenGL, GPU Resources and GLSL

13.1 Why the OpenGL 3.3 change matters

OpenGL 2.1 compatibility mode allowed simple immediate-mode calls such as `glBegin`. OpenGL 3.3 **core** instead requires defining shaders and vertex input explicitly. The GPU draws the data you upload into buffers. The current v0.4 renderer in the supplied bundle uses **GLAD 2** to obtain function pointers and GLFW to create/context-manage the window.

13.2 Two shader stages

Actual vertex-shader logic in the supplied source:

```
#version 330 core
layout(location = 0) in vec3 aPosition;
layout(location = 1) in vec3 aColour;
uniform mat4 uMVP;
out vec3 vColour;
void main() {
    gl_Position = uMVP * vec4(aPosition, 1.0);
    vColour = aColour;
}
```

`layout(location = 0)` binds attribute index 0 to a 3-component vertex position. `uniform` means one matrix value used across the draw call; `out` passes interpolated data to the next shader stage. `vec4(aPosition,1.0)` adds homogeneous coordinate $w = 1$ so translations can be represented by a matrix multiplication.

Actual fragment-shader logic:

```
#version 330 core
in vec3 vColour;
out vec4 FragColour;
uniform bool uRoundPoints;
void main() {
    if (uRoundPoints) {
        vec2 p = gl_PointCoord * 2.0 - 1.0;
        if (dot(p, p) > 1.0) discard;
    }
    FragColour = vec4(vColour, 1.0);
}
```

For points, `gl_PointCoord` gives a coordinate across the point primitive. Pixels outside a unit circle are discarded, creating round point sprites. `vec4(...,1.0)` assigns fully opaque alpha.

13.3 Compile and link shaders

```
GLuint shader = glCreateShader(type);
glShaderSource(shader, 1, &source, nullptr);
glCompileShader(shader);
glGetShaderiv(shader, GL_COMPILE_STATUS, &ok);
```

A shader **compiles** separately, then vertex and fragment shaders are **linked** into one shader program. The project checks compile/link status and throws an error containing the GPU driver's log on failure. This differs from compiling C++ files with g++: GLSL is compiled by the graphics driver at runtime.

13.4 The packed vertex layout

```
struct Vertex {
```

```
    Vector3D position;
    Vector3D colour;
};
```

vao (vertex array object) stores how vertex attributes are interpreted. vbo (vertex buffer object) owns the GPU-side byte buffer containing vertex data. The code declares the positions/colours to the GPU:

```
glBindVertexArray(vao);
glBindBuffer(GL_ARRAY_BUFFER, vbo);
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE,
    sizeof(Vertex), reinterpret_cast<void*>(offsetof(Vertex, position)));
glEnableVertexAttribArray(0);
```

- First argument `0`: GLSL attribute location 0.
- `3`: three scalar components per vertex.
- `GL_FLOAT`: underlying scalar data is `float`.
- `GL_FALSE`: don't normalise float values.
- `sizeof(Vertex)`: stride, in bytes, between consecutive vertices.
- `offsetof(...)`: byte offset from start of structure to member.
- `reinterpret_cast<void*>`: expresses the byte offset in the API's expected pointer-typed argument.

Portability caveat: for robust production code, ensure the `Vertex` layout matches the buffer attribute definition, including GLM alignment configuration. A future mesh format should consider explicit tightly packed fields or static layout assertions.

13.5 Per-frame rendering

The renderer assembles a dynamic `std::vector<Vertex>` for ground lines, active beams, broken beams, mouse spring and points. For each group, the local `drawVertices` lambda uploads current vertices:

```
glBufferData(GL_ARRAY_BUFFER,
    static_cast<GLsizeiptr>(vertices.size() * sizeof(Vertex)),
    vertices.data(), GL_DYNAMIC_DRAW);
glDrawArrays(mode, 0, static_cast<GLsizei>(vertices.size()));
```

`vertices.data()` is a pointer to contiguous C++ vector memory. OpenGL copies data into the currently bound VBO. The `GL_DYNAMIC_DRAW` usage hint indicates the data is expected to change frequently. This is functional for 28 beams; at larger scales, batch allocations and persistent buffers may become more efficient.

13.6 Why OpenGL cleanup matters

The renderer's destructor calls `glDeleteBuffers`, `glDeleteVertexArrays`, `glDeleteProgram`, destroys the GLFW window, then terminates GLFW. It makes the context current before deleting GL resources. This is **RAII** in practice: graphics resources have a controlled lifetime, instead of relying on a Python GC finalizer.

13.7 Render order and depth

The ground grid is drawn with depth testing. Beams are depth-tested as lines. The source temporarily disables depth testing when drawing node points so they remain easy to locate. That **intentionally lets rear nodes remain visible through front geometry** for interaction and can lead to ambiguous selections: a useful debug feature but not a production occlusion policy.

13.8 Future graphics improvements

Static grid VBO, more deliberate GPU buffer allocation, solid mesh skinning over the node cage, picking with depth, scalable vertex colours/strain gradients, cameras, optional ImGui inspector, and consistent frame pacing.

Sources: LearnOpenGL Hello Triangle, LearnOpenGL Camera, GLFW quick guide.

14 The GLEW Failure and GLAD 2 Migration

14.1 What actually happened on 31 October 2026

The user installed/upgraded packages using `pacman`. The first 3D renderer still used GLEW. A build of the existing sources succeeded, but running failed with:

```
SoloPhysics error: Failed to initialise GLEW: Missing GL version
```

A key debugging detail: a GLAD ZIP was referenced in instructions but **not actually present in `~/Downloads`**. The first `unzip` command failed, so running CMake in a folder named `build-glad` still built the GLEW code (`-- Found GLEW ...`). This was a *patch-not-applied problem*, independent of the underlying graphics loader error.

After the ZIP was downloaded/extracted and the project rebuilt, the user confirmed: **"it works now"**.

14.2 GLEW vs GLAD

Both are OpenGL function loaders. Modern OpenGL entry points are commonly retrieved at runtime from the active context, rather than being assumed statically exported by a system header/library. GLFW creates the window/context and provides `glfwGetProcAddress`. GLAD generates bindings for the requested API version. In this project, GLAD was selected to avoid the earlier GLEW initialization problem in the user's GLFW/Wayland setup.

> **Note:** Root-cause precision The error alone does **not** prove Wayland/GLX was the sole technical cause. There was a suspected mismatch between GLEW's initialization path and the native context on that machine. The concrete verified facts are that GLEW failed, the old code persisted while the patch was absent, and the GLAD build worked after the patch was applied.

14.3 GLAD 2, not GLAD 1

The v0.4 source uses:

```
#include <glad/gl.h>
#include <GLFW/glfw3.h>

// Only after glfwMakeContextCurrent(window):
const int loadedVersion = gladLoadGL(glfwGetProcAddress);
if (loadedVersion == 0 || !GLAD_GL_VERSION_3_3)
    throw std::runtime_error("Failed to load OpenGL 3.3 functions with GLAD");
```

`<glad/gl.h>` and `gladLoadGL(...)` are the GLAD 2 API used here; don't substitute `<glad/glad.h>` and `gladLoadGLLoader(...)` examples from GLAD 1.

Correct order: initialize GLFW → choose 3.3 core hints → create window → make context current → load GLAD → compile shader → create VAO/VBO → draw. The context must exist and be current when GLAD obtains entry points.

14.4 Current CMake method

The file in the supplied ZIP pins an upstream GLAD Git commit and uses `FetchContent`:

```
include(FetchContent)
FetchContent_Declare(
    glad
    GIT_REPOSITORY https://github.com/Dav1dde/glad.git
    GIT_TAG 658f48e72aee3c6582e80b05ac0f8787a64fe6bb
    SOURCE_SUBDIR cmake
    GIT_SHALLOW FALSE
)
```

```
FetchContent_MakeAvailable(glad)
glad_add_library(glad_gl_core_33 STATIC REPRODUCIBLE API gl:core=3.3 EXTENSIONS NONE)
```

This runs a generator (requires Python/Jinja2) and creates a CMake library target `glad_gl_core_33`. The pinned revision makes source selection repeatable; generated code and toolchains can still vary across environments. It downloads from GitHub on first configure unless cached.

14.5 Debug checklist

```
# Did the source actually change?
grep -nE 'GLEW|glad|glfw' CMakeLists.txt
# Is the new include present?
grep -nE 'glew.h|glad/gl.h|gladLoadGL' src/rendering/Renderer.cpp
# Configure cleanly in a NEW directory
cmake -S . -B build-glad -G Ninja
cmake --build build-glad
./build-glad/solo_physics
```

This case is a good lesson: **a new build directory does not change the source code or dependencies by itself**. Confirm patch extraction before debugging runtime loader behaviour.

References: GLAD upstream repository, GLAD CMake API, GLFW guide, CMake FetchContent.

15 Line-by-Line CMake and Linking Guide

15.1 The actual supplied `CMakeLists.txt`

The current reference file can be opened in `Source Snapshots/v0.4-GLAD/CMakeLists.txt`. Its key lines:

```
cmake_minimum_required(VERSION 3.23)
project(SoloPhysics LANGUAGES C CXX)
set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)
find_package(glm CONFIG REQUIRED)
find_package(glfw3 CONFIG REQUIRED)
include(FetchContent)
FetchContent_Declare(glad
    GIT_REPOSITORY https://github.com/Dav1dde/glad.git
    GIT_TAG 658f48e72aee3c6582e80b05ac0f8787a64fe6bb
    SOURCE_SUBDIR cmake
    GIT_SHALLOW FALSE
)
FetchContent_MakeAvailable(glad)
glad_add_library(glad_gl_core_33 STATIC REPRODUCIBLE API gl:core=3.3 EXTENSIONS NONE)
add_executable(solo_physics
    src/main.cpp
    src/physics/PhysicsWorld.cpp
    src/rendering/Renderer.cpp)
target_include_directories(solo_physics PRIVATE ${CMAKE_CURRENT_SOURCE_DIR}/include)
target_compile_definitions(solo_physics PRIVATE GLFW_INCLUDE_NONE)
target_link_libraries(solo_physics PRIVATE glm::glm glfw glad_gl_core_33)
target_compile_options(solo_physics PRIVATE -Wall -Wextra -Wpedantic)
```

15.2 Decode it one line at a time

Command	What it does	Python analogy
`cmake_minimum_required(...)`	minimum CMake needed to understand this build	minimum tool version
`project(... LANGUAGES C CXX)`	enable C and C++ compilation; GLAD generates C source	packaging metadata + compilers
`CMAKE_CXX_STANDARD 20`	request C++20	choosing syntax/runtime target
`CMAKE_CXX_EXTENSIONS OFF`	prefer standard C++ over GNU-specific C++ language mode	strict portability configuration
`find_package(glm CONFIG REQUIRED)`	locate installed GLM CMake package target	resolve external dependency
`find_package(glfw3 CONFIG REQUIRED)`	locate GLFW package and target	resolve external dependency
`include(FetchContent)`	load built-in CMake dependency helper	import build script functionality
`FetchContent_Declare`	describe upstream Git source and revision	pin package source URL + commit
`FetchContent_MakeAvailable`	fetch/add dependency to configure	obtain/install build dependency
`glad_add_library(...)`	generate GLAD bindings/library	generate compile-time bindings

Command	What it does	Python analogy
`add_executable`	declare target and source files	define application build
`target_include_directories`	where `#include` looks for local headers	interpreter import search path analogy
`target_compile_definitions`	define preprocessor flag	build-time configuration flag
`target_link_libraries`	link the app to required libraries	native linking, not `import`
`target_compile_options`	enable useful warnings	lint/checker options

15.3 CMake language syntax is not C++

`set(X 20)` has parentheses but is a CMake *command*, not a C++ function. Values are usually **strings/arguments** in the CMake language; variable interpolation uses `${VARIABLE}`, not Python f-strings. `PRIVATE/PUBLIC/INTERFACE` determine whether usage requirements propagate to depending targets, not the C++ access modifiers inside a class.

15.4 Compile versus link errors

A missing `#include` file is a **preprocessing/compilation** problem. A function declared but never compiled into any linked object can cause `undefined reference` at **link** time. Missing `glfw` or generated `glad_gl_core_33` linkage is also a linking/configuration issue. Failure after the binary launches and prints a GLFW/GLEW/GLAD error is a **runtime** issue.

15.5 Rebuild best practices

```
# Normal incremental build
cmake --build build-glad

# Reconfigure when CMakeLists.txt changes
cmake -S . -B build-glad -G Ninja

# Prefer fresh independent directory to troubleshoot cache issues
cmake -S . -B build-test -G Ninja
cmake --build build-test
```

Do not indiscriminately delete a directory that might contain something you placed there manually. Generated build directories can generally be recreated; keep source code outside them.

See official CMake FetchContent documentation, GLAD CMake definition.

16 Real Errors, Diagnostics and Lessons

16.1 Debugging timeline

Symptom	Observed cause / diagnosis	Fix	General lesson
Changing damping seemed ineffective	A local `main.cpp` edit wasn't saved before build	Save, then rebuild; print config values	Verify inputs and build freshness
Bottom nodes sat at ground with positive vertical velocity	Repeated tiny penetration plus restitution	Small-speed resting threshold	Contact impulses require persistent resting treatment
Clicking seemed to do nothing	Small picking radius / unclear mouse-to-world state	Print cursor world coords; increase radius; show selected node	Inspect each link in input→selection→force chain
Dragging not acting through physics	Node force assigned before `resetForces()`	Add callback inside step between reset and integration	Put state updates in correct stage
Beams never turned orange/broke	`plasticDeformationEnabled=0`, `breakStrain=100` remained set	Enable plasticity, lower test thresholds, anchor a node	Verify actual configuration, not the intended one
`i was not declared in this scope`	`world.nodes[i]` written outside `for(std::size_t i...)`	Move fixed-node `continue` inside indexed loop	Scope is lexical; `continue` is loop-only
Copying instructions into `Renderer.cpp` caused invalid C++	Markdown explanatory text/code fences were pasted into `.cpp`	Replace file with clean source	Code blocks ≠ prose instructions
`Failed to initialise GLEW: Missing GL version`	Runtime loader/context incompatibility suspected	Migrate loader to GLAD 2	Distinguish loader from GPU/shader/physics
CMake still said Found GLEW after naming build folder `build-glad`	Patch ZIP absent; `unzip` failed; source still used GLEW	Actually download/extract patch, verify `grep` results	Directory names do not change source

16.2 Reproducible debugging funnel

```
flowchart TD
 A[Does it compile?] -->|No| B[Read first compiler error]
 A -->|Yes| C[Does it link?]
 C -->|No| D[Check missing targets/source definitions]
 C -->|Yes| E[Does window/context initialise?]
 E -->|No| F[Check GLFW / loader / GPU versions]
 E -->|Yes| G[Are physics numbers valid?]
 G -->|No| H[Check dt, mass, forces, contact, strain]
 G -->|Yes| I[Do pixels match state?]
 I -->|No| J[Check shaders, VAO/VBO, matrices, depth]
 I -->|Yes| K[Test input and interactivity]
```

16.3 What `ninja: no work to do` actually means

It means Ninja believes its declared inputs/outputs are up-to-date; it does **not** prove your program uses the source you *intended* to copy. Verify path, file contents, CMake configure output and the executable invoked. Use `cmake --build build-glad --verbose` when necessary to inspect compiler/linker commands.

16.4 Warnings versus errors

The v0.4 build log included `warning: variable 'steps' set but not used`. Warnings typically do not stop compilation with the current options; they should still be addressed because they can reveal dead code or mistaken state handling. `-Wall -Wextra -Wpedantic` increases diagnostic visibility.

16.5 Useful SoloLinux commands

```
pwd
find include src -maxdepth 3 -type f | sort
rg -n 'GLEW|glad|externalForceCallback|updateDamage' include src CMakeLists.txt
cmake --build build-glad --verbose
./build-glad/solo_physics
git diff --stat
git diff -- src/main.cpp
```

`rg` is ripgrep; use regular `grep -R` if unavailable. The `git diff` commands inspect uncommitted changes.

16.6 The rule: measure before changing

When debugging a simulator, log meaningful invariants: node count, beam count, node mass, dt, position, velocity, force, rest length, strain, broken count and plastic-history count. **Change one variable at a time**. A powerful diagnostic order is: numerical state → force signs → picking → rendering. Never use just the visual cube as a correctness oracle.

17 Tests, Learning Exercises and Worked Answers

17.1 A. Regression test matrix

Test	Setup	Expected trend/criterion
Gravity-only drop	unconnected 1 kg node, no ground impact	$v_y \approx -9.81t$, $y \approx y_0 - \frac{1}{2}gt^2$; discretisation causes small error
Zero-force isolation	one node, gravity zero	position and velocity stay unchanged
Equal/opposite beam loads	2 free nodes linked by one stretched spring	internal forces sum to approximately zero
Hanging spring equilibrium	anchor 3 m, rest 1 m, $m = 1$, $k = 100$	settled height approximately **1.9019 m**
Damping comparison	compare $c = 5$ versus $c = 0.6$	smaller damping oscillates longer
Square pre-impact	4 nodes, equal masses, initial square	common downward acceleration before contact
Resting ground	square after impact and settling	bottom $y = 0$, tiny v_y after correction
Plasticity disabled	pull beam above `yieldStrain`	rest length stays unchanged, history flag false
Plasticity enabled	pull above yield without breaking	rest length changes; history flag becomes true
Fracture	pull past `breakStrain`	`broken=true`, no spring force in future steps
GLAD startup	OpenGL 3.3 driver/context	logs a loaded version; shader program links
3D picking	orbit around cube and click a node	selected node index remains valid

17.2 B. Exercise 1 — Translate Python to C++

Python:

```python
mass = 1.0
position = [0.0, 2.0, 0.0]
velocity = [0.0, 0.0, 0.0]
```

Exercise: create a `Node` in C++ and set the same state. **Answer:**

```cpp
Node node;
node.mass = 1.0f;
node.position = Vector3D{0.0f, 2.0f, 0.0f};
node.velocity = Vector3D{0.0f};
```

17.3 C. Exercise 2 — Calculate spring force

A 1 m rest beam is 1.02 m long, $k = 650$ N/m, axial relative velocity 0.1 m/s, $c = 12$ N·s/m. Find scalar force. **Answer:**

$$F = 650(0.02) + 12(0.1) = 13 + 1.2 = 14.2\,\text{N}.$$

This assumes `direction` points A→B and positive relative velocity means endpoints are separating.

17.4 D. Exercise 3 — Interpret the lambda

```
auto fn = [&](PhysicsWorld& world) {
    if (!grabbedNode) return;
    world.nodes[*grabbedNode].force += Vector3D{1, 0, 0};
};
```

Explain why the compiler allows access to `grabbedNode` even though it is not a parameter. **Answer:** `[&]` captures used enclosing local variables by reference; `fn` can read the latest `grabbedNode`, assuming its lifetime remains valid. `std::optional` presence is tested before dereferencing.

17.5 E. Exercise 4 — Derive static equilibrium

Change the hanging-mass spring to $k = 200$ N/m, $m = 1$ kg, same anchor/rest length. Predict position. **Answer:** extension $mg/k = 0.04905$ m, so

$$y_{eq} = 3 - (1 + 0.04905) = 1.95095\,\text{m}.$$

17.6 F. Exercise 5 — Why 28 beams?

For eight distinct nodes, an edge between every unordered pair creates $8 \times 7/2 = 28$ beams. This is the complete graph K_8. Not all 28 are necessary for a rigid braced cube; the current topology is a simple heavily reinforced learning structure.

17.7 G. Exercise 6 — Debug a missing force

You add `world.nodes[3].force += F;` just before calling `world.step(dt)`, but the node doesn't respond. Where does it go wrong? **Answer:** `resetForces()` zeros it inside the timestep; supply it via `externalForceCallback` (or a formal persistent external-force input system) before beam solution/integration.

17.8 H. Exercise 7 — Change OpenGL safely

Explain why these lines must be ordered:

```
glfwMakeContextCurrent(window);
int loadedVersion = gladLoadGL(glfwGetProcAddress);
```

Answer: OpenGL entry-point lookup depends on a current context. Trying to load before binding a context may fail, even if the compiler and linker are happy.

17.9 I. Suggested exercise roadmap

1. Write a tiny C++ test executable that compiles only `PhysicsWorld.cpp` plus tests; no renderer.
2. Compare analytical and simulated oscillator periods over multiple timesteps.
3. Record total energy (kinetic + spring + gravitational) with no damping/ground and explain integration drift.
4. Visualise active-beam strain as a continuous gradient while keeping plastic history a distinct overlay.
5. Expose material parameters in a configuration file rather than hardcoding each scenario.
6. Add mouse-input replay so regression tests don't depend on how quickly you move the mouse.

Each task depends on concepts already encountered in the project; keep a lab note recording hypothesis, method, result and discrepancies.

18 Current Limits and a Path Toward Vehicle Simulation

18.1 Present capabilities (confirmed by source/conversation)

- 3D point-mass nodes and axial spring–damper beams.
- External gravity; fixed anchor nodes.
- Semi-implicit Euler, fixed-step application loop.
- Simplified flat-ground node contact and restitution.
- Yield-triggered rest-length change, broken-beam flags.
- OpenGL 3.3 line/point rendering, orbit/pan/zoom, screen-space picking and mouse spring dragging.
- Native GLFW context with GLAD 2; confirmed launched successfully after migration.

18.2 Not yet established

A working 3D cube display is **not** the same as validated soft-body vehicle dynamics. Current limitations include: simplistic contact, no self-collision, no continuous collision detection, no proper constraints solver, no beam bending/buckling/torsion, no traction or suspension/wheels, no vehicle drivetrain, no contact patches, no material calibration, no deformable surface meshes, no broad-phase spatial partitioning, no replay-determinism guarantee, and no numerical performance benchmarks for thousands of beams.

18.3 Suggested engineering order

Phase	Priority	Why
1	Unit/regression testing	Guard physics correctness while changing architecture
2	Separate simulation library, scenario tests, UI	Easier to maintain/test than one large `main.cpp`
3	More robust ground contact and friction	Avoid velocity clamping artefacts
4	Better strain/damage history	Separate elastic/plastic/total strain; test energy and break criteria
5	Collision geometry and broad phase	Objects can actually crash into each other
6	Vehicle chassis + basic suspension	First vehicle-like dynamics
7	Surface mesh attached to node cage	Visible deformation beyond line segments
8	Constraint solvers, possibly XPBD	Address stiffness/time-step coupling and robust contacts
9	Profiling and scalability	Support realistic node/beam counts

An eventual switch to **XPBD** or another constraint approach is a research direction, not an assertion that the current spring formulation is wrong. XPBD specifically targets problems of stiffness depending on timestep/solver iteration count; see Macklin et al., XPBD paper.

18.4 Research questions to answer experimentally

1. How large can `stiffness` become at 240 Hz before noticeable instability?
2. How does the restitution threshold change the energy budget?
3. Is fracture triggered by instantaneous strain, accumulated history, stress, or energy release?

4. Does the rendering pick the nearest *visible* node, or just nearest in screen-space among all nodes?
5. How many nodes and beams can be simulated at 240 Hz on the target hardware?
6. How will a vehicle's tyres/suspension interact with the same contact system?

18.5 Source discipline

Official BeamNG descriptions are a **conceptual inspiration** for node–beam simulation, but the SoloPhysics code remains its own simplified educational implementation. Do not infer BeamNG internals from matching terms or copy parameter units without verification.

19 Websites, Sources and Attribution

19.1 Provenance statement — important

This bibliography has **two distinct classes of evidence**:

- **Project/author evidence (P):** user-supplied code, build logs, outputs, and conversation milestones. Those sources establish what SoloPhysics *actually did*.
- **External learning/reference (R):** authoritative websites checked while preparing this documentation (31 Oct 2026) or explicitly recommended/discussed during development. These explain third-party APIs and algorithms but are **not proof** that every page was personally consulted at the moment the original code was authored. The earlier assistant did not preserve an exhaustive browser-history ledger; therefore **an exhaustive list of every website previously visited cannot honestly be certified**.

This distinction avoids retroactively claiming websites were used when the evidence only shows they are relevant/recommended.

19.2 Direct project evidence

ID	Evidence	Use
P1	Conversations and recorded milestone notes, 28–31 Oct 2026	Design decisions, progression, suspended-mass and square-drop measurements
P2	User's pasted `main.cpp` and `Renderer.cpp` excerpts from 2D iterations	Actual C++ syntax, callbacks and earlier debugging
P3	CLI outputs for hanging mass, square drop, and improved contact	Numerical results (1.9019 m and resting square)
P4	User's build logs, 31 Oct 2026: missing patch ZIP, GLEW failure	Loader transition diagnosis
P5	`SoloPhysics-v0.4-GLAD-upgrade.zip` bundle available in the conversation	Verified v0.4 `CMakeLists.txt`, `main.cpp`, `Renderer.hpp`, `Renderer.cpp`
P6	User statement, 31 Oct 2026, "it works now"	GLAD renderer runs on user's machine

Source completeness limit: the current live repository and its newest local `PhysicsWorld.cpp` were not read in their entirety for this handbook. Source snapshots are the supplied GLAD upgrade bundle, not a Git clone of the user's project.

19.3 Official documentation and educational sources

ID	Website / precise URL	Why it matters	Status
R1	BeamNG: Soft-body Physics	Nodes, massless beams, axial constraints, deformation/breaking	verified reference; conceptual inspiration
R2	BeamNG: Introduction to JBeam	Structure, node/beam modelling, orientation	verified reference

ID	Website / precise URL	Why it matters	Status
R3	GLFW Getting Started	Window/context creation, rendering loop, GLAD loader ordering	verified reference
R4	GLFW Input Guide	Mouse/key polling, callbacks and window user pointers	verified reference
R5	GLAD upstream repository	Generator and GLAD 2 function loading	verified reference; exact repository pinned by project
R6	GLAD CMake library definition	`glad_add_library(... API gl:core=3.3)`	verified reference
R7	CMake FetchContent	Fetch/build upstream dependency	verified reference
R8	GLM Manual	`glm::vec3`, `glm::lookAt`, `glm::perspective`, `glm::project/unProject`	verified reference
R9	LearnOpenGL: Hello Triangle	VBO, VAO, shaders and draw calls	verified educational reading
R10	LearnOpenGL: Camera	View matrices and camera motion	verified educational reading
R11	Gaffer on Games: Fix Your Timestep!	Fixed-step accumulator rationale	verified educational reading
R12	cppreference: std::optional	Value-or-no-value state for selection/drag	verified C++ reference
R13	cppreference: std::function	Stored external force callback	verified C++ reference
R14	LearnCpp: Introduction to lambdas	Captures, anonymous callable objects	verified learning reference
R15	Macklin, Müller and Chentanez: XPBD (2016)	Future constraint-solver research; not current algorithm	verified primary paper

All external links were checked or retrieved during this documentation pass on **31 October 2026**. The exact content of external pages may change later. For authoritative specifications, prefer upstream docs; for tutorials, compare explanations against upstream APIs and compiler output.

19.4 Licence/attribution awareness

SoloPhysics uses third-party libraries. GLM, GLFW, GLAD, OpenGL and CMake are distinct projects with their own documentation, licensing and APIs. This handbook paraphrases their concepts and contains small original instructional examples and user-project excerpts. It does **not** reproduce substantial portions of external tutorials or articles.

19.5 Sources mentioned in original planning but not implemented

Early planning discussed position-based dynamics (PBD/XPBD) as a possible future strategy. It is not the solver currently in SoloPhysics. BeamNG references describe conceptual inspiration but the project has no claim of implementing the complete BeamNG algorithm.

20 C++ Syntax Cheat Sheet for Python Programmers

20.1 Read this before opening `Renderer.cpp`

C++ notation	Plain English	Python rough analogy
`#include <file>`	make declarations from header available	`import` (compile-time, not runtime)
`#pragma once`	don't include same header twice in translation unit	no direct equivalent
`namespace solophysics`	scope for project symbols	module/namespace
`::`	qualify a scope	`module.name`
`.member`	member of a value/reference	`obj.attr`
`ptr->member`	member through a pointer	`obj.attr` after pointer dereference
`&` after a type	reference / alias	implicit object reference, not identical
`&x` expression	take address of x	no everyday equivalent
`*ptr` expression	dereference pointer	no everyday equivalent
`*opt`	retrieve contained optional value	unwrap `Optional`
`const T&`	read-only borrowed alias	immutable-access view analogy
`T&&`	rvalue reference	advanced, not yet needed
`auto`	compiler infers a concrete type	inferred typing, but static
`constexpr`	compile-time value when possible	`Final` isn't exact equivalent
`std::vector<T>`	dynamically sized contiguous sequence	`list[T]`
`std::array<T,N>`	fixed-size array	fixed-length tuple analogy
`std::size_t`	unsigned size/index type	`int` for indices
`std::optional<T>`	maybe T; maybe no value	`T | None`
`std::function<R(Args...)>`	stored callable with signature	`Callable[[Args], R]`
`[](...) {...}`	lambda, no captures	`lambda` (but C++ allows block body)
`[&](...) {...}`	lambda captures used locals by reference	closure captures variable bindings
`[=](...) {...}`	lambda captures used locals by value	closure capture snapshot analogy
`[this]`	capture current object pointer	bound method analogy
`for (auto& x : xs)`	iterate by reference	`for x in xs:`
`for (i=0; i<n; ++i)`	index loop	`for i in range(n):`
`if (!flag)`	if not true	`if not flag:`
`&&`, `||`	logical AND, OR	`and`, `or`
`true`, `false`	boolean values	`True`, `False`
`nullptr`	null pointer	`None` (not the same type)
`{0,1,2}`	aggregate/brace initialisation	list/tuple literal analogy
`std::cout << x`	write stdout	`print(x)`
`std::cerr << x`	write stderr	`print(x, file=sys.stderr)`
`throw` / `catch`	exception handling	`raise` / `except`
`static_cast<T>(x)`	explicit conversion	`T(x)` (not exact)
`sizeof(T)`	number of bytes for C++ object type	`struct.calcsize` analogy

C++ notation	Plain English	Python rough analogy
`offsetof(T,member)`	member byte offset	C ABI `ctypes` analogy
`R"TAG(...)TAG"`	raw string literal	`r"..."` but with delimiter
`~Renderer()`	destructor	`__del__` analogy, but timing differs
`= delete`	disable a function/operation	no direct equivalent

20.2 Special vector syntax

```cpp
using Vector3D = glm::vec3;
Vector3D p{1.0f, 2.0f, 3.0f};
Vector3D v{0.0f};
Vector3D moved = p + v * 0.1f;
float distance = glm::length(moved - p);
float alignedSpeed = glm::dot(v, glm::normalize(p));
```

These operations are built by GLM's overloaded operators, analogous to NumPy array arithmetic—not automatic features of built-in C arrays.

20.3 Common traps

- = assigns; == compares.
- & is reference in a declaration but address-of in many expressions.
- * multiplies numbers and dereferences pointers/optional values depending on context.
- `[]` is indexing for a container and capture syntax for a lambda.
- `std::vector` references can be invalidated by reallocation.
- `float` and `double` are not Python's single ordinary float type; literals ending in `f` are `float`.
- Always check the *first* compiler error before the cascade that follows.

20.4 Further reading

03 - C++ for a Python Developer • 04 - Data Model and Source Architecture • 13 - Modern OpenGL and GLSL • LearnCpp • cppreference